

STUDY GUIDE · FROM ZERO TO RUNNING IT YOURSELF

PostgreSQL Backups You Can Trust — Study Guide

Every tool explained for beginners: PostgreSQL, WAL and point-in-time recovery, backup types, pgBackRest, S3 object storage, restore drills, Docker Compose, supercronic, Prometheus, Pushgateway and Grafana, security and GitHub Actions, plus hands-on labs, a glossary and 25 interview questions.

Sufyan Ahmad · DevOps Engineer

<https://github.com/sufyanahmadkamboh/sufyan-devops-postgres-backup-drills>

Contents

- 0. The big picture
- 1. PostgreSQL basics
- 2. WAL, archiving and point-in-time recovery
- 3. Backup types: full, differential, incremental
- 4. pgBackRest
- 5. Object storage (S3)
- 6. Restore drills
- 7. The Docker Compose stack
- 8. Scheduling with supercron
- 9. Monitoring: Pushgateway, postgres_exporter, alerts and Grafana
- 10. Security
- 11. GitHub Actions
- 12. How everything fits together
- 13. Hands-on labs
- Glossary
- Interview questions

Each chapter: what the tool is, why this project uses it, how it works, where it is integrated, commands to try, common mistakes, and questions with answers.

0. The big picture

The problem, without jargon

Imagine you keep a **spare key** to your house at a friend's place. You feel safe. Then one day you lock yourself out, go to your friend, and discover the key doesn't fit: the lock was changed two years ago. The spare key existed, but it was **never tested**.

Database backups often fail in exactly this way:

- The backup job ran every night and said "success", but the files cannot be restored.
- The backup is fine, but nobody knows the restore takes 9 hours.
- The backup is from last night, and the mistake happened at 4 pm, so a whole day of orders is gone.
- The backups are encrypted, and the only copy of the key was on the server that just died.
- The backups were stored next to the database, and the same attacker deleted both.

The idea of this project

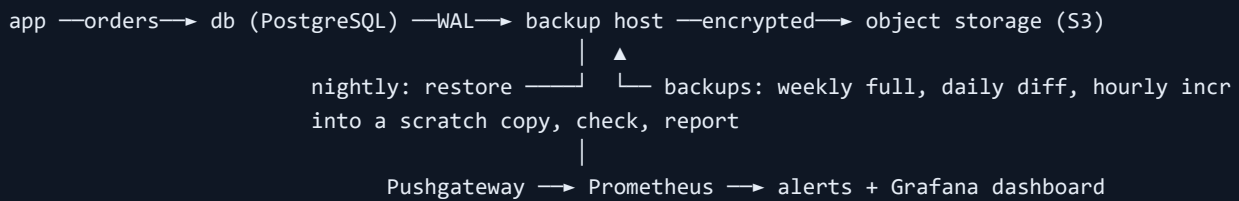
Treat backups like a **fire drill**: practise the emergency regularly, so that on the real day everyone knows it works and how long it takes.

Everyday picture	In this project
A diary in which the shop writes down every sale, line by line	PostgreSQL's WAL (write-ahead log)
Photocopying the whole diary once a week	A full backup
Copying only the pages that changed since the last copy	Differential and incremental backups
Sending each finished diary page to a safe deposit box immediately	WAL archiving to object storage (S3)
A safe deposit box in another building, which only one trusted person can open	A separate backup host with the only storage key, and storage the database cannot even reach
The box is locked and the contents are in code	Encryption of every backup file
Every night someone rebuilds the shop's records from the box and checks nothing is missing	The nightly restore drill
"Put the records back exactly as they were at 15:59, just before the mistake"	Point-in-time recovery (PITR)
A noticeboard showing when the last drill passed	Prometheus alerts and a Grafana dashboard

What the system does, in one breath

A small shop application writes orders into **PostgreSQL**. Every change is recorded in the WAL. The WAL is sent at least every minute, through a dedicated **backup host**, into **encrypted object storage**. The backup host also takes full, differential and incremental backups on a schedule. Every night it **restores the latest backup into a scratch copy**, checks that no order is missing and nothing is corrupted, measures how long it took, and

reports the result. If anything goes wrong (archiving stops, a backup fails, a file in storage is damaged, a drill fails), an **alert** fires.



Two numbers that matter

- **RPO (recovery point objective):** how much recent data you can afford to lose. Here: changes are archived at least every 60 seconds, so in a disaster you lose at most about a minute.
- **RTO (recovery time objective):** how long you can afford to be down while restoring. The drill **measures** this every night instead of guessing.

Where to go next

Chapter 1 explains the database itself. If you already know PostgreSQL, skim it and go to chapter 2, where the real magic (the WAL) is.

1. PostgreSQL basics

What is it?

PostgreSQL ("Postgres") is an open-source **relational database**: a program that stores data in **tables** (rows and columns) and lets applications read and change it with **SQL**. It is one of the most widely used databases in production, from start-ups to banks.

Why this project uses it

Backups only matter when the data matters, and the data that matters most in most companies lives in a relational database. PostgreSQL also has the features that make professional backups possible: the write-ahead log, continuous archiving and point-in-time recovery (chapter 2).

Alternatives: MySQL/MariaDB (similar ideas: binary log, Percona XtraBackup), managed databases such as Amazon RDS or Cloud SQL (the provider takes the backups, but you still need to test restores), document databases such as MongoDB. The concepts in this guide (full + incremental backups, a log of changes, restore drills) apply to all of them.

How it works: the five ideas you need

1. Server, database, table, row

One **PostgreSQL server** (also called a *cluster* in PostgreSQL language) holds several **databases**. This project has a database called `shop` with one table, `orders`. Each order is a **row**.

2. Transactions and commits

A **transaction** is a group of changes that happen all together or not at all. When the application inserts an order, PostgreSQL **commits** it: from that moment the order is guaranteed to survive a crash. That guarantee comes from the WAL (chapter 2).

3. Roles (users) and privileges

PostgreSQL controls access with **roles**. This project creates:

Role	Can do	Used by
<code>postgres</code>	everything (superuser)	PostgreSQL itself, pgBackRest on the database host
<code>app</code>	owns <code>shop</code> , reads and writes <code>orders</code>	the shop application (<code>loadgen</code>)
<code>monitor</code>	read-only, member of <code>pg_monitor</code> , <code>SELECT</code> on <code>orders</code>	<code>postgres_exporter</code> and the restore drill's comparison queries

Giving every program its **own** role with only the rights it needs is the *least privilege* principle.

4. The data directory

Everything PostgreSQL stores lives in one folder on disk, the **data directory** (`PGDATA`). In the official PostgreSQL 18 image it is `/var/lib/postgresql/18/docker`. A physical backup is, in essence, a consistent copy of this folder plus the WAL needed to make it consistent.

5. Recovery and promotion

When PostgreSQL starts from a restored backup, it is first in **recovery**: it replays WAL to bring the copy up to date. When it reaches the end (or a target time), it is **promoted**: it becomes a normal read-write database.

`select pg_is_in_recovery()` tells you which state it is in.

Where it is integrated

The database runs from `image/Dockerfile`, which starts from the official image and adds pgBackRest:

```
FROM postgres:18.6

ARG PGBACKREST_VERSION=2.59.2-1.pgdg13+1
```

On the first start (empty volume), `image/initdb/10-shop.sh` creates the roles, the database and the table:

```
CREATE ROLE app LOGIN PASSWORD :'app_password';
CREATE ROLE monitor LOGIN PASSWORD :'monitor_password' IN ROLE pg_monitor;
CREATE DATABASE shop OWNER app;
...
CREATE TABLE orders (
  id          bigserial PRIMARY KEY,
  created_at  timestampz NOT NULL DEFAULT now(),
  customer    text       NOT NULL,
  amount      numeric(10,2) NOT NULL CHECK (amount >= 0)
);
```

The passwords come from Docker **secrets** (files), never from the code. The `created_at` column is important for this project: the restore drill and the point-in-time recovery test use it to know which orders *should* exist after a restore.

The application, `image/bin/loadgen`, inserts an order every half second, like real traffic:

```
psql -XAtq -v ON_ERROR_STOP=1 -c \
  "insert into orders (customer, amount) values ('customer-' || (random() * 999)::int, round((random() * 200)::numeric, 2))"
```

Try it

```
# open a SQL prompt on the live database (as the postgres superuser)
docker compose -f deploy/compose.yaml exec -u postgres db psql -d shop

# inside psql:
select count(*), max(created_at) from orders;      -- run it twice: the count grows
\du                                                -- list roles
select pg_is_in_recovery();                        -- f = normal read-write database
show data_directory;
\q
```

Common mistakes

- **Copying the data directory with `cp` while PostgreSQL runs.** The copy is inconsistent and may not start. Use a backup tool that coordinates with PostgreSQL (pgBackRest, `pg_basebackup`).
- **Using the superuser for applications.** If the application is hacked, the attacker owns everything. Use separate roles.
- **Thinking a dump (`pg_dump`) is enough for big, busy databases.** A dump is a logical snapshot of one moment: it cannot give you point-in-time recovery, and restoring large dumps is slow.

Check yourself

1. What is the difference between a database and a table?
2. Why does this project use a separate `monitor` role instead of `postgres` for monitoring?
3. What does "promoted" mean for a restored database?

▼ Answers

1. A database is a container of tables (and other objects); a table holds rows of one kind of data, for example orders.
2. Least privilege: monitoring only needs to read statistics and the `orders` table, so if its password leaks, nobody can change or delete data.
3. It finished recovery (replaying WAL) and became a normal read-write database.
`pg_is_in_recovery()` returns `f`.

2. WAL, archiving and point-in-time recovery

What is it?

The **WAL (write-ahead log)** is PostgreSQL's diary. Before PostgreSQL changes any data file, it first writes a record of the change into the WAL. If the server crashes, PostgreSQL replays the WAL on the next start and loses nothing that was committed.

WAL archiving copies every finished WAL file somewhere safe. **Point-in-time recovery (PITR)** uses a base backup plus the archived WAL to rebuild the database **as it was at any moment you choose**, to the second.

Why this project uses it

A nightly backup alone means you can lose up to a day of orders. With continuous WAL archiving, the backup repository receives every change within about a minute. That is the difference between "we lost today's sales" and "we lost the last minute".

PITR is also the cure for **human mistakes**. If someone runs `DROP TABLE orders` at 15:00:03, you restore to 15:00:02 and the table is back with every order up to that second.

Alternatives: streaming replication to a standby server (protects against hardware failure, but a `DROP TABLE` is replicated instantly too, so it is not a backup), `pg_dump` every night (no PITR), storage snapshots (fast, but usually not point-in-time and not tested).

How it works

WAL segments

The WAL is written in files called **segments**, 16 MiB each, with names like `00000001000000000000000005`:

```
00000001 00000000 00000005
timeline "log" segment
```

When a segment is full (or forced to switch), PostgreSQL calls the `archive_command` for it. Only when that command succeeds is the segment considered archived; until then PostgreSQL keeps it on its own disk and retries.

archive_timeout: bounding data loss

A quiet database might take hours to fill 16 MiB, and changes in an unfinished segment are not archived yet. `archive_timeout=60` forces a segment switch at least every 60 seconds when there were changes. That gives an RPO of about one minute.

Recovery targets

A restore copies a base backup, then replays archived WAL: - **to the end** (`--type=default`): as recent as possible, used by the restore drill - **to a time** (`--type=time --target="2026-10-03 15:00:02+00"`): stops just before the first commit after that time, used to undo a mistake

Timelines

After a point-in-time recovery, the database's history **forks**: the old future (with the mistake) and the new one. PostgreSQL gives the new branch a new **timeline** number (1 → 2) and writes a small `.history` file to the archive, so WAL from the two branches is never mixed up. You can see it in segment names: `00000002...`.

pg_stat_archiver

PostgreSQL counts archived and failed attempts in the view `pg_stat_archiver`. This project's monitoring reads it to know whether archiving works (chapter 9).

Where it is integrated

`deploy/compose.yaml`, the `db` service:

```
command:
  - postgres
  - -c
  - wal_level=replica
  - -c
  - archive_mode=on
  - -c
  - archive_command=pgbackrest --stanza=main archive-push %p
# Send a WAL segment to the backup host at least every 60 s, even when traffic is low:
# this bounds how much recent data a restore can lose.
  - -c
  - archive_timeout=60
```

`%p` is replaced by the path of the finished segment. `pgbackrest archive-push` sends it to the backup host (chapter 4).

Point-in-time recovery is scripted in `scripts/pitr.sh`:

```
log "1/4 Archiving the latest changes"
if sql "select pg_switch_wal()" >/dev/null 2>&1; then
  on_backup pgbackrest --stanza=main check >/dev/null || die "WAL could not be archived; refusing to restore"
  ...
compose run --rm --no-deps -e ROLE=restore db --type=time "--target=$target" --target-action=promote
```

Step 1 is easy to forget and very important: the changes just before the mistake are still in the **current**, **unarchived** segment. `pg_switch_wal()` closes that segment and `pgbackrest check` waits until it is in the archive. Without it, the last minute before the mistake would be lost.

Try it

```
# see how archiving is doing
docker compose -f deploy/compose.yaml exec -u postgres db psql -d shop -c \
  "select archived_count, failed_count, last_archived_wal, last_archived_time from pg_stat_archiver"

# force a segment switch and watch last_archived_wal change a few seconds later
docker compose -f deploy/compose.yaml exec -u postgres db psql -d shop -c "select pg_switch_wal()"

# the WAL files in the repository
docker compose -f deploy/compose.yaml exec -u postgres backup pgbackrest --stanza=main repo-ls --recurse archive
```

The full "drop a table and get it back" exercise is lab 3 in [Hands-on labs](#).

Common mistakes

- **Archiving without a base backup.** WAL alone is useless; you need a backup to replay it onto.
- **No `archive_timeout`.** On a quiet database, "continuous" archiving may be hours behind.
- **Ignoring archive failures.** When `archive_command` keeps failing, WAL piles up on the database disk until it is full and the database stops. Alert on it (this project does: `WalArchivingFailing`).
- **Restoring to a time without archiving the current segment first.** The minutes just before the target are lost.
- **Choosing a target time in the wrong time zone.** Always write the offset (`+00`).

Check yourself

1. Why is a replica (standby server) not a backup?
2. What does `archive_timeout=60` guarantee, and what does it cost?
3. Why does PostgreSQL create a new timeline after a point-in-time recovery?

▼ Answers

1. It copies every change immediately, including mistakes like `DROP TABLE` . A backup lets you go back in time.
2. At most about 60 seconds of committed changes are not yet archived (the RPO). The cost: more, partly empty WAL files when traffic is low (they compress very well).
3. The database's history forks at the target. The new timeline keeps WAL from the old future (after the mistake) and the new future apart, so a later restore never mixes them.

3. Backup types: full, differential, incremental

What is it?

Type	Copies	Size	Restore needs
Full	every file	large	just this backup (+ WAL)
Differential (diff)	files changed since the last full	medium, grows during the week	the full + this diff
Incremental (incr)	files changed since the last backup of any type	small	the full + the latest diff + every incr after it

All three are **physical** backups: copies of the database files, not SQL dumps.

Why this project uses all three

It is a trade-off between backup cost and restore speed:

- Full backups every hour would be safe but huge and slow.
- One full backup and only incrementals for months would be small, but a restore would have to chain hundreds of pieces.

The classic schedule used here: **weekly full, daily differential, hourly incremental**. A restore then needs at most one full, one diff and a handful of incrementals, plus the WAL since the last of them.

Alternatives: only full backups (simplest, fine for small databases), only incrementals with periodic "synthetic fulls" (some tools), storage snapshots.

How it works

Block incremental and bundles

Newer pgBackRest versions can store only the **changed blocks** of a file, not the whole file (`repo1-block=y`). That makes incrementals of large tables much smaller. **Bundling** (`repo1-bundle=y`) packs many small files into a few larger objects, which matters for object storage (fewer requests, lower cost).

Retention

`repo1-retention-full=2` keeps the last **two** full backups and everything that depends on them. When a third full backup finishes, the oldest full, its diffs and incrementals, and the WAL only they needed are deleted (`expire`). Two fulls means you can go back one to two weeks.

Backup labels

Each backup has a label that shows its type and parents:

```
20261003-020928F      full
20261003-020928F_20261003-021255I  incremental on top of that full
```

Where it is integrated

The schedule is in `image/crontab`:

```
# Weekly full, daily differential, hourly incremental: restore = 1 full + 1 diff + a few incr + WAL.
0 1 * * 0      run-backup full
0 1 * * 1-6    run-backup diff
30 * * * *     run-backup incr
```

The options are written into the backup host's pgBackRest configuration by `image/bin/entrypoint`:

```
repo1-retention-full=${RETENTION_FULL:-2}
repo1-bundle=y
repo1-block=y
compress-type=zst
```

`image/bin/run-backup` runs one backup and reports the result to Prometheus, success or not:

```
pgbackrest --stanza="$STANZA" --type="$type" backup
rc=$?
...
backup_last_run_success $(( rc == 0 ? 1 : 0 ))
```

Try it

```
scripts/backup.sh incr      # take an incremental now
scripts/backup.sh diff
scripts/status.sh           # pgbackrest info: every backup, its size and what it depends on
```

In the `info` output, compare `database size` with `repo1: backup size`: an incremental of a quiet database is tiny.

Common mistakes

- **Deleting "old" fulls by hand.** Diffs and incrementals depend on them. Let `expire` do it.
- **Retention too short for your needs.** With `retention-full=2` and weekly fulls you cannot go back three weeks. Decide how far back you must be able to go, then set retention.
- **Only checking that backups run.** A backup that runs is not a backup that restores. That is chapter 6.

Check yourself

1. Restoring Thursday afternoon with this schedule needs which backups?
2. Why is `repo1-bundle=y` useful on object storage?
3. What happens to the WAL of a full backup that retention removes?

▼ Answers

1. Sunday's full, Thursday's 01:00 differential, the incrementals after it up to the afternoon, then the archived WAL since the last incremental.

2. Object storage charges and waits per request; thousands of small files become a few large objects, so backups and restores are faster and cheaper.
3. WAL needed only by expired backups is removed too, so the repository does not grow forever.

4. pgBackRest

What is it?

pgBackRest is an open-source backup and restore tool made for PostgreSQL. It takes full, differential and incremental backups, archives WAL, restores (also to a point in time), verifies checksums, compresses, encrypts, and stores everything on local disks or object storage (S3, Azure, GCS).

Why this project uses it

It is the most complete free tool for production PostgreSQL backups: parallel, resumable, block-incremental, with built-in encryption and a `verify` command. One tool covers backups, WAL archiving and restores, so there is one configuration and one repository to reason about.

Alternatives: WAL-G (similar idea, popular on Kubernetes), **Barman** (EDB's tool, server-based), `pg_basebackup` + a custom WAL archive script (built into PostgreSQL, but you write retention, encryption and verification yourself), managed services (RDS snapshots).

How it works

Stanza

A **stanza** is pgBackRest's name for "one PostgreSQL cluster and its backups". This project's stanza is `main`. `stanza-create` prepares the repository; `check` proves that WAL archiving reaches it.

Repository

The **repository** (`repo1`) is where backups and WAL go. Here it is an S3 bucket (`repo1-type=s3`), encrypted (`repo1-cipher-type=aes-256-cbc`) and compressed with `zstd`.

Database host and backup host

Production setups often run pgBackRest on a **dedicated backup host** (also called repository host):

	Database host (<code>db</code>)	Backup host (<code>backup</code>)
runs	PostgreSQL + <code>pgbackrest server</code>	<code>pgbackrest server</code> + the scheduler
knows the S3 key and the encryption passphrase	no	yes
can reach object storage	no	yes
sends WAL with	<code>archive-push</code> → backup host	stores it in S3
takes backups	no	yes, reading the database's files through the db host's server

So if the database server is compromised, the attacker finds no storage credentials and cannot delete the backups.

TLS server and mutual TLS

The two hosts talk through pgBackRest's built-in **TLS server**. Both sides present certificates signed by the project's private CA (**mutual TLS**). `tls-server-auth=db=main` on the backup host means: "a client whose certificate says `CN=db` may use stanza `main`". Nothing else gets in.

Asynchronous archiving

With `archive-async=y`, `archive-push` returns quickly and a background process sends WAL in batches from a local **pool** directory. If the backup host or the storage is down, WAL waits in the queue (up to `archive-push-queue-max`) and is sent once it is back.

Restore options used here

- `--delta`: compare checksums and copy only files that differ (fast when the data directory is mostly intact).
- `--type=time --target=...`: point-in-time recovery.
- `--archive-mode=off`: the restored copy must **not** archive WAL into the real repository (important for drills).
- `--recovery-option="restore_command=..."`: how the restored database fetches WAL (`archive-get`).

verify

`pgbackrest verify` reads every backup file and WAL segment in the repository and checks it against the checksum stored when it was written. It finds damaged or missing files long before you need them.

Where it is integrated

The configuration is generated at container start by `image/bin/entrypoint`, from environment variables and Docker secrets. The database host (`render_db`):

```
[global]
repo1-host=${BACKUP_HOST:-backup}
repo1-host-type=tls
repo1-host-cert-file=${CERTS}/db.crt
archive-async=y
compress-type=zst
archive-push-queue-max=${ARCHIVE_QUEUE_MAX:-4GiB}
tls-server-auth=${BACKUP_HOST:-backup}=${STANZA}

[${STANZA}]
pg1-path=${PG_PATH}
```

The backup host (`render_backup`) holds the repository options, including the secrets:

```

repo1-type=s3
repo1-s3-bucket=${S3_BUCKET:-pgbackrest}
repo1-s3-endpoint=${S3_ENDPOINT:-s3}
repo1-storage-ca-file=${CERTS}/ca.crt
repo1-s3-key=$(cat /run/secrets/s3_access_key)
repo1-cipher-type=aes-256-cbc
repo1-cipher-pass=$(cat /run/secrets/repo_cipher_pass)
...
[${STANZA}]
pg1-host=${DB_HOST:-db}
pg1-host-type=tls

```

The files end up mode `0640`, owned by `postgres`. On the first start, `image/bin/init-stanza` creates the stanza, proves archiving works and takes the first full backup:

```

pgbackrest --stanza="$STANZA" check
log "WAL archiving works"
...
log "repository is empty: taking the first full backup"
run-backup full

```

Restores of the live database use the `restore` role of the same image (`ROLE=restore`), started by `scripts/pitr.sh`:

```

exec gosu postgres pgbackrest --stanza="$STANZA" --delta --log-level-console=info restore "$@"

```

Try it

```

# what is in the repository
docker compose -f deploy/compose.yaml exec -u postgres backup pgbackrest --stanza=main info

# prove archiving works end to end (switches WAL and waits for it to arrive)
docker compose -f deploy/compose.yaml exec -u postgres backup pgbackrest --stanza=main check

# check every file in the repository against its checksum
scripts/verify.sh

# the generated configuration (note: the db host has no S3 key and no passphrase)
docker compose -f deploy/compose.yaml exec db cat /etc/pgbackrest/pgbackrest.conf

```

Common mistakes

- **Losing the encryption passphrase.** Without it, the backups are random bytes. Store it in a password manager or a secrets vault, outside the servers (lab 6 shows what happens).
- **Letting the restored copy archive WAL.** A test restore that archives into the same stanza can corrupt the real history. Use `--archive-mode=off`.
- **Restoring while PostgreSQL still runs.** `pgBackRest` refuses (`unable to restore while PostgreSQL is running`), and rightly so. Stop the database first; make sure it shut down cleanly.
- **Never running `verify`.** Damaged files are found only when you need them.

Check yourself

1. Why does the database host not have the S3 key?
2. What does `tls-server-auth=db=main` allow?
3. What problem does `archive-async=y` solve when storage is down?

▼ Answers

1. If the database server is compromised, the attacker cannot use the key to read, change or delete the backups. Only the backup host can reach storage.
2. Clients presenting a certificate (signed by the project CA) with the name `db` may use the stanza `main` on this TLS server. Every other client is refused.
3. WAL is queued locally in the spool directory and `archive_command` keeps succeeding quickly, so the database keeps running. Once storage is back, the queue is sent.

5. Object storage (S3)

What is it?

Object storage stores files ("objects") in **buckets** and is used over HTTP(S) with an API. **S3** is Amazon's object storage, and its API has become the standard: many products speak "S3-compatible" (SeaweedFS, Ceph, Garage, Cloudflare R2, Wasabi, Hetzner Object Storage...).

Why this project uses it

Backups must live **somewhere else** than the database: another machine, ideally another building or region. Object storage is cheap, practically unlimited, very durable, and has features made for backups (versioning, object lock). pgBackRest writes to it natively.

The lab runs **SeaweedFS** in a container, so the whole project works offline and costs nothing. In production you change the endpoint to AWS S3 (or any S3-compatible service); nothing else changes.

Why not MinIO? For years MinIO was the default local S3. In 2025 it stopped publishing community binaries and container images, went into maintenance mode, and the repository was archived in 2026. A project that is supposed to be usable in production should not start on an abandoned component, so this one uses SeaweedFS, which is actively maintained.

Alternatives: Garage (small, single binary), Ceph RGW (large clusters), RustFS (newer), the cloud providers' own storage.

How it works

Buckets, objects, keys

- A **bucket** is a top-level container: here `pgbackrest`.
- An **object** is a file inside it, with a path-like name: `main/archive/main/18-1/.../000000010000000000000005-....zst`.
- Programs authenticate with an **access key** and a **secret key** (like a user name and password). Each key can be limited to certain buckets and actions.

Path-style vs virtual-hosted URLs

AWS uses `https://bucket.s3.amazonaws.com/object` (virtual-hosted style). Local services usually need `https://s3:8443/bucket/object` (**path style**), hence `repo1-s3-uri-style=path`.

HTTPS with a private CA

pgBackRest only talks to S3 over HTTPS and checks the certificate. In the lab, `scripts/setup.sh` creates a small **private certificate authority** and a certificate for the host name `s3`; pgBackRest trusts that CA through `repo1-storage-ca-file`.

Where it is integrated

`deploy/compose.yaml`:

```
s3:
  networks: [storage]
  image: chrislusf/seaweedfs:4.48
  command:
    - server
    - -dir=/data
    - -master.telemetry=false
    - -s3
    - -s3.port=8333
    - -s3.port.https=8443
    - -s3.cert.file=/certs/s3.crt
    - -s3.key.file=/certs/s3.key
    - -s3.config=/run/secrets/s3_identities
```

`-s3.config` points to the access rules generated by `scripts/setup.sh`: one identity, `pgbackrest`, allowed to read, write, list and tag only the bucket `pgbackrest`:

```
"actions": ["Read:pgbackrest", "Write:pgbackrest", "List:pgbackrest", "Tagging:pgbackrest"]
```

A one-shot `s3-init` service creates the bucket if it does not exist yet (`s3.bucket.create -name pgbackrest`).

Moving to AWS S3 in production

Only the backup host's environment changes, for example:

```
environment:
  S3_ENDPOINT: s3.eu-central-1.amazonaws.com
  S3_REGION: eu-central-1
  S3_URI_STYLE: host
```

plus real keys in the secrets (or an IAM role). Turn on bucket versioning or **Object Lock** so that even a stolen key cannot delete old backups.

Try it

```
# the bucket's contents, as pgBackRest sees them
docker compose -f deploy/compose.yaml exec -u postgres backup pgbackrest --stanza=main repo-ls --recurse
backup

# HTTPS health check, trusting the project CA
docker compose -f deploy/compose.yaml exec backup curl -s --cacert /etc/pgbackrest/tls/ca.crt
https://s3:8443/healthz -o /dev/null -w '%{http_code}\n'

# without a key, the bucket is closed (expect 403)
docker compose -f deploy/compose.yaml exec backup curl -s --cacert /etc/pgbackrest/tls/ca.crt
https://s3:8443/pgbackrest/ -o /dev/null -w '%{http_code}\n'
```

Common mistakes

- **Storing backups on the same server or the same account as the database.** One incident takes both.
- **Disabling TLS checks** (`verify-tls=n`) "to make it work". Use a proper CA instead.
- **One key with full admin rights** for backups. Limit the key to one bucket.

- **No protection against deletion.** Ransomware deletes backups first. Use versioning/Object Lock and keep the key off the database host.

Check yourself

1. What is the difference between a bucket and an object?
2. Why does the lab need `repo1-s3-uri-style=path` ?
3. What would you change to move the repository to AWS S3?

▼ Answers

1. A bucket is a named container; objects are the files stored inside it, each with a key (name).
2. The local service is reached as `https://s3:8443/<bucket>/...` ; virtual-hosted style would need DNS names like `pgbackrest.s3` , which don't exist in the lab.
3. The endpoint, region and URI style on the backup host, plus real credentials (or an IAM role) and the AWS certificate chain instead of the private CA. The database host does not change at all.

6. Restore drills

What is it?

A **restore drill** is a planned, regular restore of your backups into a separate place, followed by checks that the result is complete and correct. It answers the only question that matters about backups: *can we get the data back, and how long does it take?*

Why this project uses it

"The backup job succeeded" only proves that some files were written. Real restores fail for many reasons nobody notices in daily operation: a damaged file in storage, a missing WAL segment, a changed encryption key, a version mismatch, a restore that takes ten times longer than expected. This project restores **every night**, automatically, and alerts when a drill fails or does not run. Untested backups are a guess; tested backups are a fact.

Alternatives: manual restore tests a few times a year (better than nothing, but rare and easily skipped), restoring into a staging environment as part of another job, managed "backup testing" services.

How it works: RTO and RPO

- **RTO (recovery time objective):** how long a restore may take. The drill measures the real time: download + start + WAL replay + checks. The alert `RestoreTooSlow` fires above 30 minutes.
- **RPO (recovery point objective):** how much data may be lost. The drill measures it directly: the newest order in the live database minus the newest order in the restored copy. The alert `RestoreDataLossHigh` fires above 5 minutes.

How the drill works, step by step

`image/bin/restore-drill` describes itself at the top:

```
# 1. Note the newest order in the live database (the point we should be able to get back to).
# 2. Restore the latest backup + all archived WAL from S3 into a scratch directory on this host.
# 3. Start that copy as a separate PostgreSQL (port 5433, no network, no WAL archiving).
# 4. Check it: no missing orders, how much recent data was not yet archived, no corruption (amcheck).
# 5. Throw the copy away and publish the result for Prometheus and Grafana.
```

Step 2, the restore, uses a separate config file (`drill.conf`) whose data path is `/var/lib/drill/data` , so it can never overwrite the real database:

```
pgbackrest --config="$CONF" --stanza="$STANZA" --archive-mode=off \
--recovery-option="restore_command=pgbackrest --config=$CONF --stanza=$STANZA archive-get %f \"%p\" \" \" \
restore || fail "pgbackrest restore failed"
```

Step 3 starts the copy without network access and without archiving:

```
pg_ctl -D "$DATA" -l /var/log/pgbackrest/drill-postgres.log -w -t 900 \
-o "-p $PORT -c listen_addresses='' -c unix_socket_directories=/var/run/postgresql -c archive_mode=off
-c shared_buffers=128MB" \
start
```

Step 4, the checks: - **No missing orders**: count the restored orders, then ask the live database (as the read-only `monitor` role) how many orders exist up to the highest restored id. The two numbers must be equal. - **Data loss**: newest live order time minus newest restored order time. - **No corruption**: `pg_amcheck` --`heapallindexed` reads every table and index of the copy and checks their internal structure.

```
missing_rows=$(( live_rows_upto - restored_rows ))
data_loss_seconds=$(( live_newest > restored_newest ? live_newest - restored_newest : 0 ))
...
if pg_amcheck -h /var/run/postgresql -p "$PORT" --database="$DB" --heapallindexed --no-dependent-indexes
>/tmp/amcheck.log 2>&1; then
    amcheck_ok=1
```

Step 5 always runs, even when a step fails, because of `trap cleanup EXIT`: the copy is stopped and deleted, and the result (success or failure, plus every number) is pushed to Prometheus.

What a failed drill tells you

The drill reports *which step* failed (`FAILED at step: restore from repository`). The labs show three real causes it catches: a damaged backup file, a wrong encryption key, and missing WAL.

Where it is integrated

- The script: `image/bin/restore-drill`
- The schedule, nightly at 04:00 UTC: `image/crontab` → `0 4 * * * restore-drill`
- The drill config: written by `render_backup` in `image/bin/entrypoint` (`drill.conf`)
- Manual run: `scripts/drill.sh`
- Alerts: `RestoreDrillFailed` , `RestoreDrillMissing` , `RestoreTooSlow` , `RestoreDataLossHigh` in `deploy/prometheus/rules/backups.yml`

Try it

```
scripts/drill.sh          # runs a drill now and prints each step
scripts/status.sh        # repository + the numbers of the last drill
```

Then open Grafana (<http://localhost:3000>): the top row, "Can we restore right now?", shows the result, the restore time and the data loss.

Common mistakes

- **Restoring the drill into the production data directory.** Always use a separate path (and here, a separate config file).
- **Letting the drill copy archive WAL.** It would push its own new timeline into the real repository. `--archive-mode=off` prevents it.

- **Only checking that the copy starts.** A copy can start and still miss data. Compare with the source and run `amcheck`.
- **Not alerting on missing drills.** A drill that silently stopped running is as bad as no drill (`RestoreDrillMissing`).

Check yourself

1. Why does the drill compare "orders up to the highest restored id" instead of "all orders"?
2. What is the difference between RTO and RPO? Which numbers measure them here?
3. Why does the drill use `trap cleanup EXIT`?

▼ Answers

1. The live database keeps receiving orders during and after the restore; those newer orders cannot be in the copy. Comparing up to the restored point checks for gaps without counting legitimate new orders as missing.
2. RTO is how long recovery takes (`restore_drill_duration_seconds`); RPO is how much recent data is lost (`restore_drill_data_loss_seconds`).
3. So the scratch copy is always stopped and deleted and the result is always published, also when a step fails halfway.

7. The Docker Compose stack

What is it?

Docker runs programs in **containers**: isolated processes with their own file system, built from an **image**. **Docker Compose** describes several containers, their networks, volumes and secrets in one YAML file and starts them together.

Why this project uses it

The whole system (database, backup host, storage, application, monitoring) runs on one machine with one command, identical on a laptop, in CI and on a single production VM. Compose also makes the **security design** visible in one file: who can talk to whom, who gets which secret.

Alternatives: Kubernetes (for many servers; operators such as CloudNativePG or Crunchy PGO wrap pgBackRest/Barman), plain systemd services on VMs, Ansible-managed hosts.

How it works: the services

Service	Image	Job
db	pg-backup-drills:dev (ROLE=db)	PostgreSQL 18 + pgBackRest TLS server
backup	same image (ROLE=backup)	pgBackRest TLS server, repository access, scheduler
app	same image (ROLE=app)	writes orders continuously
s3	chrislusf/seaweedfs:4.48	S3-compatible object storage (HTTPS)
s3-init	same	creates the bucket once, then exits
pushgateway	prom/pushgateway:v1.11.3	receives results of backups and drills
postgres-exporter	prometheuscommunity/postgres-exporter:v0.20.1	database metrics, including WAL archiving
prometheus	prom/prometheus:v3.15.0	collects metrics, evaluates alert rules
grafana	grafana/grafana:13.2.3	dashboard

One image, several roles. The `ROLE` variable decides what `image/bin/entrypoint` starts. The same pinned PostgreSQL and pgBackRest versions are therefore guaranteed on the database, the backup host and in restores.

How it works: the three networks

```
# Network separation: the database and the application cannot reach the backup storage at all,
# so a compromised database host cannot read, change or delete its own backups.
networks:
  data:
  storage:
  monitoring:
```

Network	Members
data	db, app, backup, postgres-exporter
storage	s3, s3-init, backup
monitoring	backup, pushgateway, postgres-exporter, prometheus, grafana

Only `backup` is on all three. The database can send WAL to the backup host, but it cannot even resolve the name `s3`.

How it works: secrets, init, health checks

- **Secrets** are files mounted at `/run/secrets/<name>`, never environment variables in the YAML and never in git. They are generated once by `scripts/setup.sh` into `.secrets/` (ignored by git).
- **init: true** runs a tiny init process as PID 1, which forwards signals and reaps child processes. The `db` container runs two processes (PostgreSQL and the pgBackRest server), so this matters.
- **stop_signal: SIGINT** on `db`: SIGINT is PostgreSQL's *fast* shutdown. The default SIGTERM means *smart* shutdown (wait for every client to disconnect), which can run past Docker's timeout and leave the database uncleanly stopped, and pgBackRest refuses to restore over a database that looks still running.
- **Health checks** and **depends_on conditions** order the start: storage healthy → bucket created → database healthy → backup host starts.

```
backup:
  networks: [data, storage, monitoring]
  ...
  secrets: [s3_access_key, s3_secret_key, repo_cipher_pass, monitor_password]
  depends_on:
    db: {condition: service_healthy}
    s3-init: {condition: service_completed_successfully}
```

Where it is integrated

- The stack: `deploy/compose.yaml`
- The image: `image/Dockerfile` and `image/bin/`
- Start and stop: `scripts/up.sh`, `scripts/down.sh` (`--purge` also deletes the data)
- Shared helpers (`compose`, `on_backup`, `sql`, `wait_for`): `scripts/lib.sh`

Try it

```

scripts/up.sh                                # first start: builds, starts, waits for the
first full backup
docker compose -f deploy/compose.yaml ps      # every service and its health
docker compose -f deploy/compose.yaml logs -f backup # stanza, backups, scheduler (Ctrl+C to stop
following)
docker compose -f deploy/compose.yaml exec db getent hosts s3 || echo "db cannot see s3"
docker compose -f deploy/compose.yaml exec backup getent hosts s3

```

Common mistakes

- **Putting every container on one network.** Then every container can reach the backups.
- **Passwords as environment variables in the compose file.** They end up in git, in `docker inspect`, in logs. Use secrets.
- **Forgetting `stop_grace_period` and the right stop signal for databases.** A killed database restarts with crash recovery, and some tools refuse to work on it.
- **`docker compose down -v` by accident.** `-v` deletes volumes, which here means the database and the whole backup repository. This project only does it with an explicit `scripts/down.sh --purge`.

Check yourself

1. Why is the `backup` service the only one on all three networks?
2. What does `init: true` do, and why does the `db` container need it?
3. Why `stop_signal: SIGINT` for PostgreSQL?

▼ Answers

1. It is the only component that must talk to the database (to back it up), to storage (to write the repository) and to monitoring (to report results). Keeping everything else apart limits what a compromised container can reach.
2. It runs a minimal `init` as PID 1 that forwards signals and reaps zombie processes. The `db` container runs PostgreSQL and the `pgBackRest` server side by side.
3. `SIGINT` triggers PostgreSQL's fast shutdown, which ends sessions and finishes cleanly within seconds; `SIGTERM` (smart shutdown) may wait too long and get killed.

8. Scheduling with supercronic

What is it?

cron runs commands on a timetable written as five fields: minute, hour, day of month, month, day of week. **supercronic** is a cron implementation made for containers: it reads a normal crontab file, runs jobs in the foreground, writes their output to the container log, and needs no root privileges.

Why this project uses it

Backups, verification, drills and metrics must run on a schedule on the backup host. Classic `cron` inside a container is awkward: it wants to run as root, daemonizes, mails output instead of logging it, and loses environment variables. `supercronic` fixes all of that, and it is a single static binary.

Alternatives: systemd timers (on a VM), Kubernetes CronJobs (one pod per run), a CI scheduler (GitHub Actions `schedule`), an external job scheduler.

How it works

```
┌ minute (0-59)
├ ┌ hour (0-23)
│ │ ┌ day of month
│ │ │ ┌ month
│ │ │ │ ┌ day of week (0 = Sunday)
30 * * * * run-backup incr      → at minute 30 of every hour
0 1 * * 0 run-backup full       → Sundays at 01:00
0 1 * * 1-6 run-backup diff     → Monday to Saturday at 01:00
```

All times are **UTC** (containers run in UTC). Each job is a small script that does one thing and reports its result to the Pushgateway, so a job that fails is visible on the dashboard and can alert.

Where it is integrated

The binary is downloaded in `image/Dockerfile`, and its SHA-256 is checked before it is used:

```
RUN curl -fsSL -o /usr/local/bin/supercronic \
    "https://github.com/aptible/supercronic/releases/download/${SUPERCRONIC_VERSION}/supercronic-
linux-amd64" \
    && echo "${SUPERCRONIC_SHA256} /usr/local/bin/supercronic" | sha256sum -c - \
    && chmod 0755 /usr/local/bin/supercronic
```

The timetable is `image/crontab`:

```
0 1 * * 0 run-backup full
0 1 * * 1-6 run-backup diff
30 * * * * run-backup incr
0 3 * * * run-verify
0 4 * * * restore-drill
* * * * * backup-metrics
```

The backup host starts it (as the `postgres` user) after the stanza is ready, in `image/bin/entrypoint` :

```
gosu postgres bash -c 'init-stanza && exec supercronic -passthrough-logs /etc/backup.crontab' &
```

The order of the night matters: the differential at 01:00, verification at 03:00 (checks it), the drill at 04:00 (restores it).

Try it

```
docker compose -f deploy/compose.yaml logs backup | grep -E 'job (succeeded|failed)' | tail
docker compose -f deploy/compose.yaml exec backup cat /etc/backup.crontab

# run any job by hand, exactly as the scheduler would
docker compose -f deploy/compose.yaml exec backup run-backup incr
docker compose -f deploy/compose.yaml exec backup backup-metrics
```

Common mistakes

- **Forgetting the time zone.** "01:00" in a container is 01:00 UTC.
- **Jobs that fail silently.** A cron job's exit code is invisible unless something records it. Here every job publishes success or failure.
- **Overlapping jobs.** Two backups at once fail (pgBackRest takes a lock). Space the schedule, as done here.
- **Downloading binaries without checking them.** Pin the version and verify the checksum.

Check yourself

1. When does `0 1 * * 1-6` run?
2. Why is the verification scheduled before the drill?
3. Why does every job push its result instead of relying on the scheduler's log?

▼ Answers

1. At 01:00 UTC from Monday to Saturday.
2. Verification reads the repository first, including the fresh differential; then the drill restores it. A verify failure explains a later drill failure.
3. Logs are not watched by anyone; metrics are. Pushing the result lets Prometheus alert when a job fails or stops running.

9. Monitoring: Pushgateway, postgres_exporter, alerts and Grafana

What is it?

- **Prometheus** collects numbers (metrics) over time by *scraping* HTTP endpoints, stores them, and evaluates **alert rules**.
- The **Pushgateway** is a small Prometheus helper for **batch jobs**: a job that runs for a minute and exits cannot be scraped, so it *pushes* its result to the Pushgateway, which Prometheus then scrapes.
- **postgres_exporter** reads PostgreSQL's statistics views and exposes them as metrics.
- **Grafana** shows metrics on dashboards.
- **promtool** is Prometheus's command-line tool; `promtool test rules` runs unit tests for alert rules.

Why this project uses it

A backup system that nobody watches decays quietly. The monitoring here answers one question: **"if the database died right now, could we get the data back?"** That means watching WAL archiving (continuous), backups (hourly/daily/weekly), the repository's integrity (nightly verify) and the drills (nightly restore).

Alternatives: pgBackRest's own `check` in a monitoring system such as Nagios/Icinga, the `pgbackrest_exporter` project, cloud monitoring (CloudWatch), healthchecks.io-style "dead man's switch" pings.

How it works: where the metrics come from

Source	Metrics	How
<code>run-backup</code>	<code>backup_last_run_success{type}</code> , duration, timestamp	pushed after every backup
<code>run-verify</code>	<code>backup_verify_success</code> , duration, timestamp	pushed after every verification
<code>restore-drill</code>	<code>restore_drill_success</code> , <code>_duration_seconds</code> (RTO), <code>_data_loss_seconds</code> (RPO), <code>_missing_rows</code> , <code>_amcheck_ok</code> ...	pushed after every drill
<code>backup-metrics</code> (every minute)	<code>pgbackrest_backup_last_timestamp_seconds{type}</code> , sizes, counts, <code>pgbackrest_stanza_status_code</code>	reads <code>pgbackrest info --output=json</code> , pushes
<code>postgres_exporter</code>	<code>pg_stat_archiver_archived_count</code> , <code>pg_stat_archiver_failed_count</code> , <code>pg_stat_archiver_last_archive_age</code>	scraped

The push helper, `image/bin/pushmetrics` , never fails the caller, because a monitoring outage must not stop a backup:

```
if ! curl -fsS --max-time 10 --data-binary @- "$url" >/dev/null; then
  echo "pushmetrics: could not push to $url" >&2
fi
exit 0
```

`image/bin/backup-metrics` is a small Python program (standard library only) that turns `pgbackrest info` into metrics. It is unit-tested against **real pgBackRest output** saved in `tests/fixtures/info-one-full.json` by `tests/test_backup_metrics.py`.

How it works: the 11 alert rules

All in `deploy/prometheus/rules/backups.yml`:

Alert	Fires when	Severity
WalArchivingFailing	archive attempts failed in the last 5 min (for 2 min)	critical
WalArchivingStale	nothing archived for more than 10 min	critical
BackupFailed	the last backup of a type failed	critical
BackupTooOld	no backup of any type for more than 2 h	critical
FullBackupTooOld	newest full backup older than 8 days	warning
RepositoryUnreadable	pgBackRest reports the stanza is not OK	critical
BackupVerifyFailed	verification found damaged or missing files	critical
RestoreDrillFailed	the last drill failed	critical
RestoreDrillMissing	no drill for 26 h	warning
RestoreTooSlow	the drill took more than 30 min (RTO)	warning
RestoreDataLossHigh	the restored copy was more than 5 min behind (RPO)	warning

Two details worth learning from:

```
- alert: WalArchivingFailing
  expr: increase(pg_stat_archiver_failed_count[5m]) > 0
```

`failed_count` only grows. Alerting on `> 0` would fire forever after the very first failure (for example during start-up, before the backup host was ready). `increase(...[5m])` asks "did it fail *recently*?".

```
expr: restore_drill_duration_seconds > 1800 and on(job) restore_drill_success == 1
```

In PromQL, `A and B` returns A's values. Written the other way round, the alert text would say "took 1s" (the value of `success`). The unit tests caught exactly this mistake.

How it works: testing the alerts

`tests/prometheus/backups.test.yml` feeds invented time series into the rules and states which alerts must (and must not) fire, and with which text:

```
- interval: 1m
input_series:
  - series: 'pg_stat_archiver_failed_count{job="postgres"}'
    values: "14x20"
alert_rule_test:
  - eval_time: 15m
    alertname: WalArchivingFailing
    exp_alerts: []
```

This one proves that old failures do **not** keep the alert firing.

How it works: the dashboard

`deploy/grafana/dashboards/backups.json`, provisioned automatically: - **Can we restore right now?** last drill PASSED/FAILED, restore time (RTO), data loss (RPO), missing orders, repository check - **Backups:** age of the newest backup per type, firing alerts, repository size, backups kept - **Continuous archiving:** seconds since the last archived WAL, archived vs failed per 5 min - **Drill history:** restore time and data loss per drill, restored orders

Try it

```
# Prometheus UI: http://localhost:9090 → Alerts tab
# Grafana:      http://localhost:3000 (anonymous viewer)

# what the Pushgateway holds right now
docker compose -f deploy/compose.yaml exec backup curl -s http://pushgateway:9091/metrics | grep -E
'^restore_drill|^backup_'

# run the alert rule tests (no stack needed)
docker run --rm -v "$PWD:/w" -w /w/tests/prometheus --entrypoint promtool prom/prometheus:v3.15.0 test
rules backups.test.yml
```

Common mistakes

- **Alerting on counters with `> 0`** instead of on their recent increase.
- **No "missing" alerts.** A job that stopped running sends no failure; alert on the *age* of the last success (`BackupTooOld`, `RestoreDrillMissing`).
- **Pushing without grouping labels.** Each backup type is pushed under its own `type` label, so the full backup's result does not overwrite the incremental's.
- **Untested alert rules.** They fail exactly when you need them. `promtool` tests are cheap.

Check yourself

1. Why does a backup job push its result instead of being scraped?
2. Why `increase(pg_stat_archiver_failed_count[5m]) > 0` and not `pg_stat_archiver_failed_count > 0`?
3. Which alert catches a drill that silently stopped running?

▼ Answers

1. It is a batch job: it runs and exits, so there is nothing to scrape afterwards. The Pushgateway keeps its last result for Prometheus.
2. The counter never goes down; a single failure long ago would keep the alert firing forever. The increase over 5 minutes means "failing now".
3. `RestoreDrillMissing` : no drill result newer than 26 hours (or none at all).

10. Security

What is it?

Backups contain **all your data**, so they are a prime target: an attacker wants to read them (data theft) or delete them (so ransomware cannot be undone). Backup security means protecting the copies at least as well as the database.

Why it matters here

Every design decision in this project answers one of three questions:

1. If someone steals a backup file, can they read it? → **encryption at rest**
2. If the database server is hacked, can the attacker destroy the backups? → **separate backup host, network separation, credentials only where needed**
3. Can someone pretend to be the database or the backup host? → **mutual TLS with a private CA**

How it works

Encryption at rest

pgBackRest encrypts every file before it leaves the backup host (`repo1-cipher-type=aes-256-cbc` with a long random passphrase). The object store only ever sees encrypted bytes. The end-to-end test downloads a raw backup file and checks that no order data is readable in it.

The passphrase is the key to everything. Lose it and the backups are useless (lab 6). Keep a copy in a password manager or secrets vault, **not** only on the backup host.

Credentials only where they are needed

Secret	db	backup	app	s3
S3 access key + secret	—	✓	—	(identity config)
repository passphrase	—	✓	—	—
postgres password	✓	—	—	—
app password	✓ (to create the role)	—	✓	—
monitor password	✓	✓ (drill comparison)	—	—

From `deploy/compose.yaml` :

```
# Dedicated backup host: holds the S3 credentials and the encryption passphrase (the database does not).
backup:
  secrets: [s3_access_key, s3_secret_key, repo_cipher_pass, monitor_password]
```

Network separation

The `db` and `app` containers are not on the `storage` network (chapter 7). Even with root on the database host, there is no route to the object store.

Mutual TLS

`scripts/setup.sh` creates a private CA and one certificate per host (`s3` , `db` , `backup`). The CA's **private key stays in** `.secrets/` and is never mounted into a container:

```
C="$ROOT_NATIVE/.secrets/certs"
CA_KEY="$ROOT_NATIVE/.secrets/ca.key" # kept OUT of certs/, which is mounted into containers
```

Inside each container, the entrypoint copies only that host's certificate and key into a private directory readable by `postgres` alone:

```
local name="${CERT_NAME:-$ROLE}" # the restore role acts as the database host
for f in "$CERTS_SRC"/ca.crt "$CERTS_SRC/$name".crt "$CERTS_SRC/$name".key; do
  [[ -f "$f" ]] && install -o postgres -g postgres -m 0600 "$f" "$CERTS/"
done
```

Least privilege inside PostgreSQL

Separate roles for the application (`app`) and monitoring (`monitor` , read-only). The drill connects to the live database only as `monitor` .

Supply chain

- **Pinned versions:** PostgreSQL 18.6, pgBackRest 2.59.2, SeaweedFS 4.48, Prometheus, Grafana and the exporters.
- **Checksums:** supercronic is downloaded with a SHA-256 check.
- **Vulnerability scan:** CI scans the image with Trivy and fails on fixable HIGH/CRITICAL vulnerabilities (chapter 11). The first scan failed: the official postgres image ships `gosu` (a tool that switches to another user), built with an old Go runtime that had 1 CRITICAL and 21 HIGH CVEs. The image now deletes it and uses a tiny shim with the same name, `image/bin/gosu` , built on `setpriv` from Debian's util-linux. Lesson: a base image you didn't write can still be your vulnerability, and scanning finds it.

Where it is integrated

- Secrets and certificates: `scripts/setup.sh` → `.secrets/` (in `.gitignore`)
- Who gets which secret, networks: `deploy/compose.yaml`
- TLS settings, private certificate copies: `image/bin/entrypoint`
- The checks: section 9 ("Security checks") of `scripts/e2e.sh`

Try it

```
# the database host's pgBackRest config has no S3 key and no passphrase (expect 0)
docker compose -f deploy/compose.yaml exec db sh -c 'grep -cE "s3-key|cipher-pass"
/etc/pgbackrest/pgbackrest.conf'

# the database cannot resolve the storage host
docker compose -f deploy/compose.yaml exec db getent hosts s3 || echo "no route to storage: good"

# only the files in .secrets/, never in git
git status --ignored | grep .secrets
```

Common mistakes

- **Backup credentials on the database server.** The first thing ransomware uses.
- **Encryption key stored next to the backups.** Encryption then protects nothing.
- **No copy of the encryption key anywhere else.** One lost server = all backups lost.
- `verify-tls=n` and **self-signed certificates without a CA.** Anyone in between can read or change traffic.
- **The CA private key inside containers.** Whoever reads it can issue themselves a trusted certificate.

Check yourself

1. Which two protections stop a hacked database server from deleting its backups?
2. Where must a copy of the repository passphrase be kept, and why?
3. Why is the CA private key not in the `certs/` folder?

▼ Answers

1. The S3 credentials are not on the database host, and the database host is not on the storage network. (In production add bucket versioning/Object Lock.)
2. Outside the servers, for example in a password manager or secrets vault. If the backup host is lost, the passphrase is needed to restore anything.
3. `certs/` is mounted into containers; anyone with the CA key could create certificates that every component trusts.

11. GitHub Actions

What is it?

GitHub Actions runs **workflows** (YAML files in `.github/workflows/`) on GitHub's machines when something happens: a push, a pull request, a schedule. A workflow has **jobs**; each job runs **steps** on a fresh virtual machine (a **runner**).

Why this project uses it

Every change is tested automatically, exactly as a reviewer would want: static checks, a security scan of the image, and the complete end-to-end test with real backups, restores and failures. A weekly scheduled run also catches problems from new base images even when nobody changes the code.

Alternatives: GitLab CI, Jenkins, CircleCI, Buildkite.

How it works: the three jobs

`.github/workflows/ci.yaml` :

Job	What it does	Fails when
static	<code>scripts/lint.sh</code> : ShellCheck, yamllint, ruff, pytest, hadolint, promtool, compose and dashboard validation	any check finds a problem
image	builds the image, scans it with Trivy	a fixable HIGH or CRITICAL vulnerability exists
e2e	<code>scripts/e2e.sh</code> on a fresh runner, then uploads <code>reports/</code>	any of the 11 sections fails

```
e2e:
  name: End-to-end (backups, drills, PITR, outage, damage, lost key)
  needs: static
  runs-on: ubuntu-24.04
  timeout-minutes: 45
  steps:
    - uses: actions/checkout@v5
    - name: Run the end-to-end test
      run: scripts/e2e.sh
```

`needs: static` means the expensive test only runs when the cheap checks passed. `if: always()` steps publish the results to the job summary and keep the reports as an artifact even when the test fails, and `scripts/down.sh --purge` always cleans up.

The end-to-end test, section by section

`scripts/e2e.sh` writes `reports/e2e-results.md`. Every number in it is measured:

1. fresh start (secrets, certificates, stack, stanza, first full backup)
2. the application writes; incremental and differential backups
3. restore drill (no missing orders, amcheck)
4. repository verification

5. accident (`DROP TABLE orders`) and point-in-time recovery, then a backup and drill on the new timeline
6. storage outage: alert fires, the database keeps working, archiving catches up, no orders lost
7. bit rot: a backup file is damaged in storage; verify and the drill catch it; alerts fire
8. lost encryption key: nothing can be restored without it
9. security checks (network separation, encryption at rest, credentials, anonymous S3 access)
10. monitoring (every series present, no alert left firing)

Static checks you may not know

- **ShellCheck** finds bugs in shell scripts (unquoted variables, `A && B || C` traps).
- **hadolint** lints Dockerfiles (pinned versions, `pipefail` in `RUN` steps).
- **ruff** lints Python.
- **yamllint** checks YAML style and syntax.

Where it is integrated

- Workflow: `.github/workflows/ci.yaml`
- Static checks, also runnable locally: `scripts/lint.sh`
- End-to-end: `scripts/e2e.sh` (resume from a section with `E2E_FROM=5 scripts/e2e.sh`)

Try it

```
scripts/lint.sh           # needs shellcheck, yamllint, ruff, pytest and Docker
scripts/e2e.sh           # the full test (deletes this stack's data first)
E2E_FROM=6 scripts/e2e.sh # only sections 6-10, on the running stack
```

On GitHub: the repository's **Actions** tab → a run → the job summary shows `e2e-results.md`.

Common mistakes

- **Testing only on the developer's laptop.** CI proves it works on a clean machine.
- **No cleanup step with `if: always()`.** Failed runs leave state behind (matters for self-hosted runners).
- **Only static checks.** They cannot tell you whether a restore works.

Check yourself

1. Why does the `e2e` job have `needs: static`?
2. What does the `image` job protect against?
3. Why does the workflow also run on a schedule?

▼ Answers

1. The end-to-end test is slow; there is no point running it when a quick check already failed.
2. Shipping an image with known, fixable HIGH/CRITICAL vulnerabilities in its packages.
3. Base images and packages change over time even when the code does not; the weekly run notices breakage early.

12. How everything fits together

This chapter follows three journeys through the system: one order, one nightly drill, and one accident.

Journey 1: one order, from insert to the safe deposit box

```
1 app (loadgen)      INSERT INTO orders ...                → db, as role app
2 db (PostgreSQL)    writes the change to the WAL, then commits → order is crash-safe
3 db                 segment full or archive_timeout (60 s) reached → calls archive_command
4 db (pgBackRest)    archive-push %p → queued in /var/spool/pgbackrest (archive-async)
5 db → backup        sends the segment over mutual TLS (cert CN=db, allowed by tls-server-
auth=db=main)
6 backup             compresses (zstd), encrypts (AES-256), uploads over HTTPS → s3 bucket pgbackrest
7 db                 pg_stat_archiver.archived_count + 1 → postgres-exporter → Prometheus
```

Files: `image/bin/loadgen`, `deploy/compose.yaml` (`archive_command`, `archive_timeout`), `image/bin/entrypoint` (both `pgBackRest` configs).

If step 6 fails (storage down), steps 1-5 keep working: segments wait in the spool, `failed_count` grows, `WalArchivingFailing` fires, and everything is sent once storage is back.

Journey 2: the nightly drill (04:00 UTC)

```
1 supercronic        runs restore-drill as postgres on the backup host
2 restore-drill       asks the live db (role monitor): newest order time
3 pgbackrest restore latest backup + WAL from s3 → /var/lib/drill/data (drill.conf, --archive-
mode=off)
4 pg_ctl start        copy on port 5433, no network, replays WAL, promotes
5 checks              count orders in copy vs live orders up to the copy's highest id → missing =
0?
                      newest live order - newest restored order → data loss
(RPO)
                      pg_amcheck --heapallindexed → corruption?
6 cleanup (trap)      stop and delete the copy; push success, RTO, RPO, counts → Pushgateway
7 Prometheus          restore_drill_success == 0 → RestoreDrillFailed; no drill in 26 h →
RestoreDrillMissing
8 Grafana              "Can we restore right now?" row
```

Files: `image/crontab`, `image/bin/restore-drill`, `deploy/prometheus/rules/backups.yml`.

Journey 3: the accident (`DROP TABLE orders`) and point-in-time recovery

```
15:00:02 last good moment      (note it: this is the target)
15:00:05 DROP TABLE orders     (the mistake; the app starts failing)
scripts/pitr.sh "2026-10-03 15:00:02+00"
1/4 pg_switch_wal() + pgbackrest check → the WAL with the last good seconds is now archived
2/4 docker compose stop app db      → fast shutdown (SIGINT), clean stop
3/4 ROLE=restore: pgbackrest restore --delta --type=time --target=... --target-action=promote
delta: only files that differ are copied; WAL is replayed up to 15:00:02 and stops
before the first commit after it (the DROP)
4/4 start db → recovery → promotion → new timeline (1 → 2); start app
then: take a new full backup; the next drill tests the new timeline
```

Files: `scripts/pitr.sh`, the `restore` role in `image/bin/entrypoint`, section 5 of `scripts/e2e.sh`.

"Remove X: what breaks?"

If you remove...	What breaks
<code>archive_timeout=60</code>	on a quiet database, recent changes may wait hours to be archived; data loss in a disaster grows
<code>archive-async=y</code>	when storage is down, every <code>archive_command</code> call waits and fails; WAL builds up and commits can slow down
<code>--archive-mode=off</code> in the drill	the scratch copy pushes its own WAL into the real repository and pollutes it
the <code>storage</code> network separation	the database and app containers can reach the object store (and its unauthenticated admin port)
<code>repo1-cipher=*</code>	backups are stored in plain text; anyone with bucket access reads all orders
the <code>pgbackrest check</code> step in <code>pitr.sh</code>	the last minute before the mistake is lost
<code>stop_signal: SIGINT</code>	slow "smart" shutdown can be killed, leaving <code>postmaster.pid</code> ; the restore then refuses to run
<code>run-verify</code>	damaged files in storage are discovered only when a restore fails
the restore drill	nobody knows whether the backups restore or how long it takes
<code>increase(...[5m])</code> in <code>WalArchivingFailing</code>	one old failure keeps the alert firing forever
<code>pushmetrics</code> always exiting 0	a Pushgateway outage would make every backup "fail"

Concept → file map

Concept	File
Image, pinned versions	<code>image/Dockerfile</code>
pgBackRest configuration (db, backup, drill)	<code>image/bin/entrypoint</code>
Stanza creation, first backup	<code>image/bin/init-stanza</code>
Backups and their metrics	<code>image/bin/run-backup</code>
Repository verification	<code>image/bin/run-verify</code>
Restore drill	<code>image/bin/restore-drill</code>
Repository metrics	<code>image/bin/backup-metrics</code>
Schedule	<code>image/crontab</code>
Database, roles, table	<code>image/initdb/10-shop.sh</code>
Services, networks, secrets	<code>deploy/compose.yaml</code>

Concept	File
Alert rules / their tests	<code>deploy/prometheus/rules/backups.yml</code> / <code>tests/prometheus/backups.test.yml</code>
Dashboard	<code>deploy/grafana/dashboards/backups.json</code>
Secrets and certificates	<code>scripts/setup.sh</code>
Point-in-time recovery	<code>scripts/pitr.sh</code>
End-to-end proof	<code>scripts/e2e.sh</code>
CI	<code>.github/workflows/ci.yml</code>

13. Hands-on labs

Start the stack once with `scripts/up.sh` (it waits for the first full backup). All commands run from the repository root, in bash (Git Bash on Windows). To save typing:

```
dc() { docker compose -f deploy/compose.yaml "$@"; }
sql() { dc exec -T -u postgres db psql -d shop -XAtc "$1"; }
```

Compare your numbers with [docs/test-results.md](#).

Lab 1: Explore

1. `dc ps` : which services are running, which are healthy? Which one has exited, and why is that fine?
2. `scripts/status.sh` : how many backups exist, of which types? What is the database size vs the backup size?
3. `sql "select count(*), max(created_at) from orders"` twice, 10 seconds apart. How many orders per second does the app write?
4. `dc exec db cat /etc/pgbackrest/pgbackrest.conf` and `dc exec backup cat /etc/pgbackrest/pgbackrest.conf`. Which one has `repo1-s3-key` ? Why?
5. Open Grafana (<http://localhost:3000>) and Prometheus (<http://localhost:9090> → Alerts).

Lab 2: Run a drill and read it

```
scripts/drill.sh
```

1. Find each of the five steps in the output.
2. How long did the restore take, and how long the whole drill (RTO)?
3. How many seconds behind the live database was the restored copy (RPO)? Why is it usually below about 60 s?
4. Run `scripts/backup.sh incr` and then the drill again. Did anything change?

Lab 3: The accident: drop a table, get it back

```
T="$(sql "select to_char(clock_timestamp() at time zone 'utc', 'YYYY-MM-DD HH24:MI:SS.US') || '+00:00'")"
BEFORE="$(sql "select count(*) from orders where created_at <= '$T'")"
echo "target=$T orders=$BEFORE"
sleep 3
sql "drop table orders" # the mistake
scripts/pitr.sh "$T" # type 'restore' to confirm
sql "select count(*) from orders where created_at <= '$T'" # must equal $BEFORE
sql "select timeline_id from pg_control_checkpoint()" # a new timeline
```

1. How long did the recovery take from start to finish?
2. Why does `pitr.sh` run `pg_switch_wal()` and `pgbackrest check` *before* stopping the database?
3. Take a new full backup (`scripts/backup.sh full`) and run a drill. Does it pass on the new timeline?

Lab 4: Storage outage

```
dc stop s3
sleep 60; sql "select archived_count, failed_count from pg_stat_archiver"
```

1. Is the application still writing orders? (Count them twice.)
2. Wait until Prometheus shows **WalArchivingFailing** (Alerts tab). How long did it take, and why not immediately? (Look at `for:` in the rule.)
3. `dc start s3`. How long until `last_archived_time` is recent again?
4. Run `scripts/backup.sh incr` and `scripts/drill.sh`. Are any orders missing?

Lab 5: Bit rot: a backup file is damaged in storage

The object store's internal filer API (port 8888) lets you replace a file directly, simulating damaged storage. Find the first bundle of the newest full backup, damage 4 KiB in the middle, and keep a copy:

```
LABEL="$(dc exec -T -u postgres backup sh -c "pgbackrest --stanza=main info --output=json | python3 -c
'import json,sys; print([b[\"label\"] for b in json.load(sys.stdin)[0][\"backup\"] if
b[\"type\"]==\"full\"][0])')")"
F="/buckets/pgbackrest/main/backup/main/$LABEL/bundle/1"
dc exec -T s3 sh -c "curl -fsS -o /tmp/orig http://localhost:8888$F && cp /tmp/orig /tmp/rot \
&& dd if=/dev/urandom of=/tmp/rot bs=4096 seek=50 count=1 conv=notrunc && curl -fsS -o /dev/null -F
file=@/tmp/rot http://localhost:8888$F"
scripts/verify.sh      # what does it report?
scripts/drill.sh       # which step fails?
```

1. Which alerts fire?
2. Put the original back: `dc exec -T s3 sh -c "curl -fsS -o /dev/null -F file=@/tmp/orig http://localhost:8888$F"`, then verify and drill again.
3. Without the nightly verify and drill, when would you have noticed?

Lab 6: The lost encryption key

```
dc exec -T -e PGBACKREST_REPO1_CIPHER_PASS=not-the-real-key backup restore-drill
scripts/drill.sh
```

1. What error does the first drill show?
2. Where would you keep the real passphrase so that losing the backup host does not lose the backups?

Lab 7: Change a parameter

Pick one: - **RPO**: change `archive_timeout=60` to `archive_timeout=300` in `deploy/compose.yaml`, run `dc up -d db`, wait 10 minutes, run a few drills. How does the data loss number change? Put it back. - **Retention**: set `RETENTION_FULL: "1"` in the `backup` service's environment, run `dc up -d backup`, take two full backups (`scripts/backup.sh full` twice) and look at `scripts/status.sh`. What did `expire` delete? Put it back.

Lab 8: Test the tests

1. In `deploy/prometheus/rules/backups.yml`, change `WalArchivingFailing` 's expression to `pg_stat_archiver_failed_count > 0`.
2. Run the rule tests:

```
bash docker run --rm -v "$PWD:/w" -w /w/tests/prometheus --entrypoint promtool prom/prometheus:v3.15.0
test rules backups.test.yml
```

3. Which test fails, and what real-life problem does it describe? Revert the change and run the tests again.
4. Bonus: in `image/bin/backup-metrics`, make `wal_segment_number` return `0` and run `python -m pytest tests`. Which test catches it?

Lab 9: The disk is gone: rebuild the database from the backups

The worst day: the database's data volume is lost (failed disk, deleted volume, new server).

```
scripts/status.sh                                # note how many backups exist
docker compose -f deploy/compose.yaml exec -u postgres db psql -d shop -Atc "select count(*) from
orders"
scripts/recover-lost-db.sh                        # type 'recover': deletes the volume, restores, replays all
WAL
docker compose -f deploy/compose.yaml exec -u postgres db psql -d shop -Atc "select count(*) from
orders"
```

Questions: 1. Read `scripts/recover-lost-db.sh`. Why does it pass `--type=default` to the restore container? (Hint: what would `docker compose run db` pass to `pgBackRest` *without* arguments?) 2. What would happen if you simply started the `db` service on an empty volume? Why can that empty database not damage the backups? (Search `docs/troubleshooting.md` for "system-id".) 3. Is the order count after recovery lower, equal or higher than before? Why can it be higher?

▼ Answers

1. Without arguments, Compose passes the service's `command`: `( postgres -c ... )` to `pgBackRest`, which fails with "option '-c' must begin with --". Any explicit option replaces that command; `--type=default` means "replay every archived change".
2. The PostgreSQL image would create a brand-new, empty database. Its system identifier differs from the one the repository was created for, so `archive-push` is refused ("system-id ... do not match") and nothing is written over the real backups.
3. Equal or higher: the script archives the latest WAL first, and the application keeps inserting orders between your count and the moment it is stopped.

Lab 10: The whole proof

```
scripts/e2e.sh                                # deletes this stack's data, then runs all 11 sections
cat reports/e2e-results.md
```

Compare each measured number with `docs/test-results.md`. Clean up with `scripts/down.sh --purge`.

Glossary

Term	Meaning
amcheck / pg_amcheck	PostgreSQL extension and tool that check tables and indexes for corruption
archive_command	the command PostgreSQL runs for every finished WAL segment; here <code>pgbackrest archive-push %p</code>
archive_timeout	forces a WAL segment switch after N seconds with changes; bounds data loss (RPO)
archive-async	pgBackRest mode that queues WAL locally and sends it in the background
backup host (repository host)	the separate server that runs backups and holds the repository credentials
bit rot	silent damage to stored data over time; found by checksums (<code>verify</code>)
block incremental	storing only changed blocks of a file, not the whole file (<code>repo1-block</code>)
bucket	a top-level container in object storage
bundle	many small files packed into one object (<code>repo1-bundle</code>)
CA (certificate authority)	the key that signs certificates; everyone trusting the CA trusts its certificates
commit	the moment a transaction becomes permanent
cron / crontab	time-based job scheduling and its file format
data directory (PGDATA)	the folder holding all of a PostgreSQL server's data
delta restore	restore that copies only files that differ from the backup
differential backup	files changed since the last full backup
Docker secret	a file mounted into a container at <code>/run/secrets/<name></code>
drill (restore drill)	a regular, automated test restore with checks
expire	pgBackRest removing backups (and WAL) beyond the retention
full backup	a copy of every database file
incremental backup	files changed since the last backup of any type
mutual TLS (mTLS)	both sides of a connection prove their identity with certificates
object storage / S3	storage of files (objects) in buckets over an HTTP API
Object Lock	S3 feature that prevents deleting or changing objects for a period
PITR (point-in-time recovery)	restoring a database to an exact moment using a backup plus WAL
promotion	a recovered database leaving recovery and becoming read-write
Pushgateway	receives metrics pushed by batch jobs, for Prometheus to scrape
pg_stat_archiver	PostgreSQL view counting archived and failed WAL segments

Term	Meaning
recovery	PostgreSQL replaying WAL to bring a copy up to date
recovery target	where recovery stops: end of WAL, a time, a transaction, a name
repository (repo1)	where pgBackRest stores backups and WAL
retention	how many backups are kept (<code>repo1-retention-full</code>)
role	a PostgreSQL user or group
RPO (recovery point objective)	the maximum acceptable data loss, measured in time
RTO (recovery time objective)	the maximum acceptable time to restore service
SeaweedFS	open-source distributed storage with an S3-compatible API; the lab's object store
segment (WAL segment)	one 16 MiB WAL file
spool	local queue directory for asynchronous WAL archiving
stanza	pgBackRest's name for one PostgreSQL cluster and its backups (<code>main</code>)
supercronic	a cron for containers
timeline	a branch in a database's history, created by point-in-time recovery
TLS server (pgBackRest)	pgBackRest's built-in service that lets hosts talk securely
verify	pgBackRest command that checks every repository file against its checksum
WAL (write-ahead log)	the log of every change, written before the data files are changed

Interview questions

25 questions this project prepares you for. Try to answer before opening the answer.

1. What is the difference between a backup that runs and a backup that restores?

▼ Answer

A job can write files successfully while the result is unusable: damaged objects, missing WAL, a lost encryption key, incompatible versions. Only a restore with checks proves it. That is why this project restores every night and alerts on failure.

2. Explain RPO and RTO with numbers from this project.

▼ Answer

RPO is the acceptable data loss: WAL is archived at least every 60 s (`archive_timeout`), and the drill measures the real gap between live and restored data. RTO is the acceptable restore time: the drill measures it every night; alerts fire above 5 minutes of data loss or 30 minutes of restore time.

3. What is the WAL and why does it make point-in-time recovery possible?

▼ Answer

Every change is written to the WAL before the data files. A base backup plus all WAL since it can be replayed to any moment, stopping just before a chosen time.

4. How would you undo an accidental `DROP TABLE` at 15:00:05?

▼ Answer

Note the last good time (15:00:02), archive the current WAL segment (`pg_switch_wal` + `pgbackrest check`), stop the app and database, run `pgbackrest restore --delta --type=time --target="... 15:00:02+00" --target-action=promote` , start the database (new timeline), start the app, take a new full backup. `scripts/pitr.sh` does exactly this.

5. Why is a streaming replica not a backup?

▼ Answer

It replicates every change immediately, including deletions and corruption. It protects against hardware failure, not against mistakes, attacks or software bugs.

6. Full, differential, incremental: what does a restore need for each?

▼ Answer

Full: just itself. Differential: the full plus the diff. Incremental: the full, the latest diff, and every incremental after it. Plus WAL since the last backup in all cases.

7. Why use a dedicated backup host instead of running backups on the database server?

▼ Answer

Storage credentials and the encryption passphrase stay off the database server, and the database server cannot reach storage at all. A compromised database host cannot read or delete its backups.

8. What does `archive-async` do and what happens when storage is down?

▼ Answer

WAL is queued in a local spool and sent in the background, so `archive_command` returns quickly. When storage is down the queue grows (up to `archive-push-queue-max`), the database keeps running, `WalArchivingFailing` fires, and the queue drains when storage returns.

9. Why must a test restore use `--archive-mode=off`?

▼ Answer

Otherwise the restored copy, after promotion, would archive WAL of a new timeline into the production repository and pollute it.

10. What does the restore drill check, beyond "the database starts"?

▼ Answer

That no orders are missing up to the restored point (compared with the live database through a read-only role), how far behind live the copy is (RPO), and that `pg_amcheck --heapallindexed` finds no corruption in tables and indexes.

11. How do you detect silent damage in backup storage?

▼ Answer

`pgbackrest verify` reads every file and compares checksums. This project runs it nightly and alerts on failure; the lab damages a file in storage to prove it is caught (and that the drill fails too).

12. What happens if you lose the repository encryption passphrase?

▼ Answer

The backups cannot be read at all. Keep a copy outside the servers (password manager, secrets vault), and test it: the lab shows a drill with the wrong key failing.

13. Why SeaweedFS and not MinIO for a local S3?

▼ Answer

MinIO stopped publishing community images in 2025 and the project was archived in 2026. SeaweedFS is maintained and S3-compatible; in production the endpoint points to AWS S3 or another provider without other changes.

14. How would you protect backups against ransomware?

▼ Answer

Credentials only on a separate backup host, network separation, encryption, bucket versioning or Object Lock (immutable for N days), a second copy in another account or region, and drills so you know the restore works.

15. Why does a batch job push metrics to a Pushgateway?

▼ Answer

It runs briefly and exits, so Prometheus cannot scrape it. The Pushgateway holds its last result. Group by labels (here `type`) so results don't overwrite each other.

16. Why alert on `increase(pg_stat_archiver_failed_count[5m]) > 0` rather than `failed_count > 0`?

▼ Answer

Counters never decrease; one old failure would keep the alert firing forever. The increase means "failing right now".

17. How do you alert on a job that silently stopped running?

▼ Answer

Alert on the age of the last success, not on failures: `BackupTooOld`, `FullBackupTooOld`, `RestoreDrillMissing` (including `absent()` when there was never a result).

18. How do you test alert rules?

▼ Answer

`promtool test rules` with input series and expected alerts, including cases that must not fire. In this project the tests caught a rule whose summary showed the wrong value because of how PromQL works.

19. What is mutual TLS and how is it configured between the hosts here?

▼ Answer

Both client and server present certificates signed by a trusted CA. `pgBackRest`'s TLS server is configured with `tls-server-auth=<client-CN>=<stanza>`, so only the `db` certificate may use stanza `main` on the

backup host and vice versa.

20. Why is the CA private key kept out of the mounted certificate directory?

▼ Answer

Anyone who reads it can issue certificates every component trusts. Containers only need their own certificate, key and the CA certificate.

21. Why does the database container stop with SIGINT?

▼ Answer

SIGINT is PostgreSQL's fast shutdown. The default SIGTERM (smart shutdown) waits for clients and may be killed by Docker's timeout, leaving `postmaster.pid` ; `pgBackRest` then refuses to restore over it.

22. What is a timeline and when do you see a new one?

▼ Answer

A branch in the database's history. After a point-in-time recovery the database continues on a new timeline (1 → 2), so WAL from the abandoned future and the new one never mix.

23. How would you move this setup to AWS?

▼ Answer

Point the backup host at an S3 bucket (endpoint, region, host-style URIs, IAM role or keys), enable versioning/Object Lock and lifecycle rules, run the database and backup host on separate instances or use the same image on ECS/Kubernetes, and keep the drills and alerts unchanged.

24. What would you monitor first if you could monitor only three things?

▼ Answer

WAL archiving failures/age (continuous protection), age of the newest successful backup, and the result and age of the last restore drill.

25. How does CI prove the system works?

▼ Answer

Static checks (ShellCheck, hadolint, yamllint, ruff, pytest, promtool), a Trivy scan of the image, and an end-to-end test on a fresh runner: backups, drill, verify, PITR after a `DROP TABLE` , a storage outage, a damaged file, a lost key, security and monitoring checks, with every number measured.